

Learning C Through Small Projects

This document records my progress learning the C programming language through a structured series of small, hands-on projects.

Each lesson introduces one major concept and builds on the previous exercises.

Learning roadmap

Lesson	Main concept	Project / focus	Status
01	Compilation, Make, and <code>main()</code>	Hello World	Completed
02	<code>argc</code> , <code>argv</code> , and loops	Command-line arguments	Completed
03	Variables, arithmetic, and conditionals	Simple calculator	Completed
04	Loops	Multiplication table	Completed
05	Functions	Refactor calculator	Completed
06	Header files	Split project into modules	In progress
07	Strings	Tiny echo clone	Planned
08	File I/O	Count lines in a file	Planned
09	Structs	Contact catalog	Planned
10	Dynamic memory	Allocation and lifetime	Planned
11	Pointer fundamentals	Addresses and arrays	Planned
12	Memory addresses and offsets	Hex addresses and <code>sizeof</code>	Planned
13	Pointer arithmetic	Manual memory traversal	Planned
14	Raw memory and buffers	Byte-buffer inspection	Planned
15	Memory layout and alignment	Struct layout and padding	Planned
16	Memory safety	Bounds, lifetime, and UB	Planned
17	Binary file I/O	Read and inspect raw bytes	Planned
18	Combined fundamentals	Tiny Unix utility	Planned

The later memory-focused lessons build progressively from addresses and pointer fundamentals to offsets, pointer arithmetic, raw byte buffers, alignment, buffer layouts, allocation lifetime, and memory safety. These lessons will explicitly distinguish defined behavior from implementation-defined, unspecified, and undefined behavior.

Completed lessons

Lesson 1: Hello World

This lesson introduced the basic structure of a C program, including `#include`, the `main()` function, `printf()`, compilation with GCC, and automated builds using Make.

Lesson 2: Command-line arguments

This lesson introduced `argc` and `argv`, which allow a C program to receive arguments from the command line. It also introduced program exit codes and basic input validation.

Lesson 3: Simple calculator

This lesson introduced variables, numeric types, arithmetic operators, integer and floating-point division, and protection against division by zero.

Lesson 4: Loops and control flow

This lesson introduced:

- for loops
- while loops
- if-else if-else conditional branching
- break to exit a loop
- continue to skip the remainder of an iteration
- return to exit the current function

The exercises demonstrated that statements execute sequentially. Control-flow statements only affect code that has not yet executed.

For example:

```
if (i == 5) {  
    continue;  
}  
  
printf("%d\n", i);
```

Equivalent Python exercises were created to compare universal programming concepts with language-specific syntax.

Lesson 5: Functions

This lesson introduced defining and calling functions, passing parameters, and using return values. The calculator was refactored into focused helper functions for addition, subtraction, multiplication, integer division, remainder, and decimal division while retaining the division-by-zero guard in `main()`. A Python comparison exercise reinforced that decomposing a program into functions is a universal design concept, even though C and Python express types, division, and program structure differently.

Comparative exercises

Selected C exercises are recreated in other languages when the comparison strengthens understanding.

Python provides comparison with a higher-level general-purpose language. PowerShell will provide comparison with scripting, structured objects, pipelines, and Windows automation.

C remains the primary learning track.

Current position

Translation unit	Knows about	Produces
<code>calculator-main.c</code> plus its headers	Function declarations and <code>main()</code>	<code>calculator-main.o</code>
<code>calculator.c</code> plus its header	Declarations and function definitions	<code>calculator.o</code>

Doing hands-on exercises and learning to reason through Make dependency behavior, compilation/linking stages, and runtime program logic. Also covered topics such as:

- `-c` means: compile and assemble, but stop before linking.
- `-o filename` means: name the output file filename.
- `.o` is the conventional extension for an object file.
- `.h` is the conventional extension for a header file.